

TASK 4 -RESPONSIBLE AI AND MODEL INTERPRETATION USING SHAP ANALYSIS REPORT

Intern :

Komaroji Sai Swathika

Internship :

InternSpark AI Internship Program

Role :

Artificial Intelligence Intern

INTRODUCTION

Responsible Artificial Intelligence helps ensure that machine learning systems are transparent, fair, and understandable. In this project, a machine learning model was developed using the Breast Cancer dataset and Random Forest algorithm. SHAP analysis was applied to understand feature importance and explain model predictions. Bias analysis was also performed to evaluate fairness and mitigation strategies were proposed.

OBJECTIVES

- To build and train a machine learning model
 - To evaluate model performance
 - To analyze feature importance using SHAP
 - To identify possible bias in predictions
 - To provide mitigation techniques for responsible AI
-

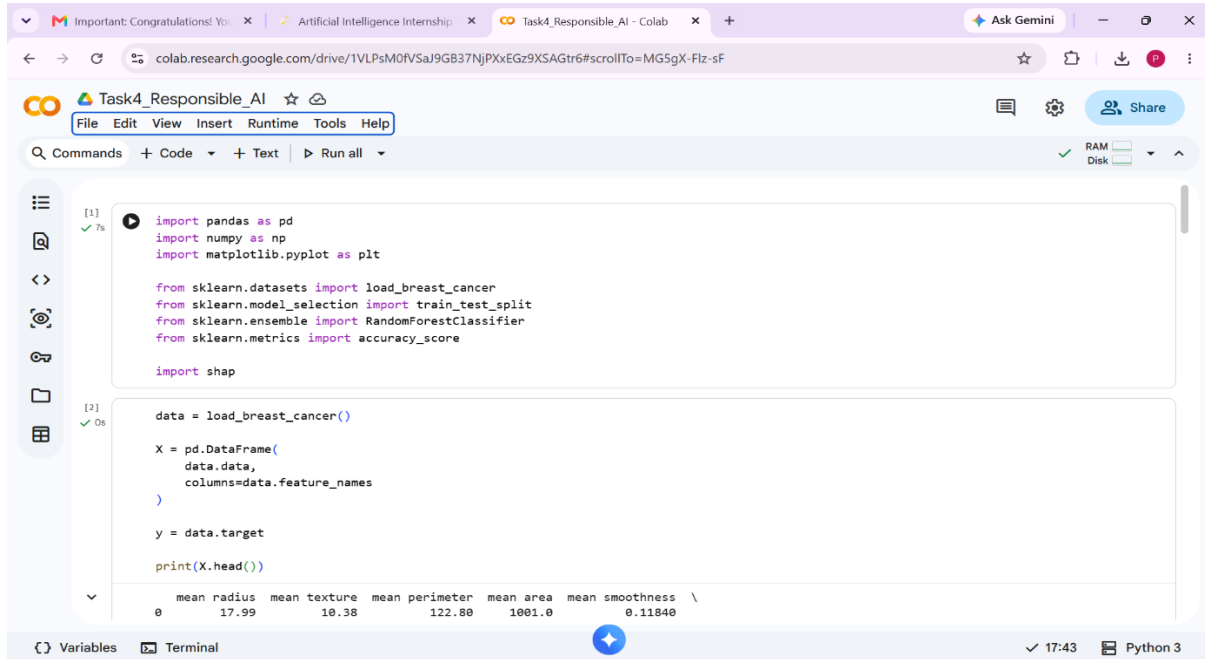
TOOLS AND TECHNOLOGIES USED

- Python
 - Google Colab
 - Pandas
 - NumPy
 - Matplotlib
 - Scikit-learn
 - SHAP
-

IMPLEMENTATION

1. Importing Required Libraries

The necessary Python libraries were imported to perform machine learning tasks. Pandas and NumPy were used for data handling, Matplotlib for visualization, Scikit-learn for model building, and SHAP for model interpretation.



The screenshot shows a Google Colab notebook titled "Task4_Responsible_AI". The code cell [1] contains the following imports:

```
[1] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

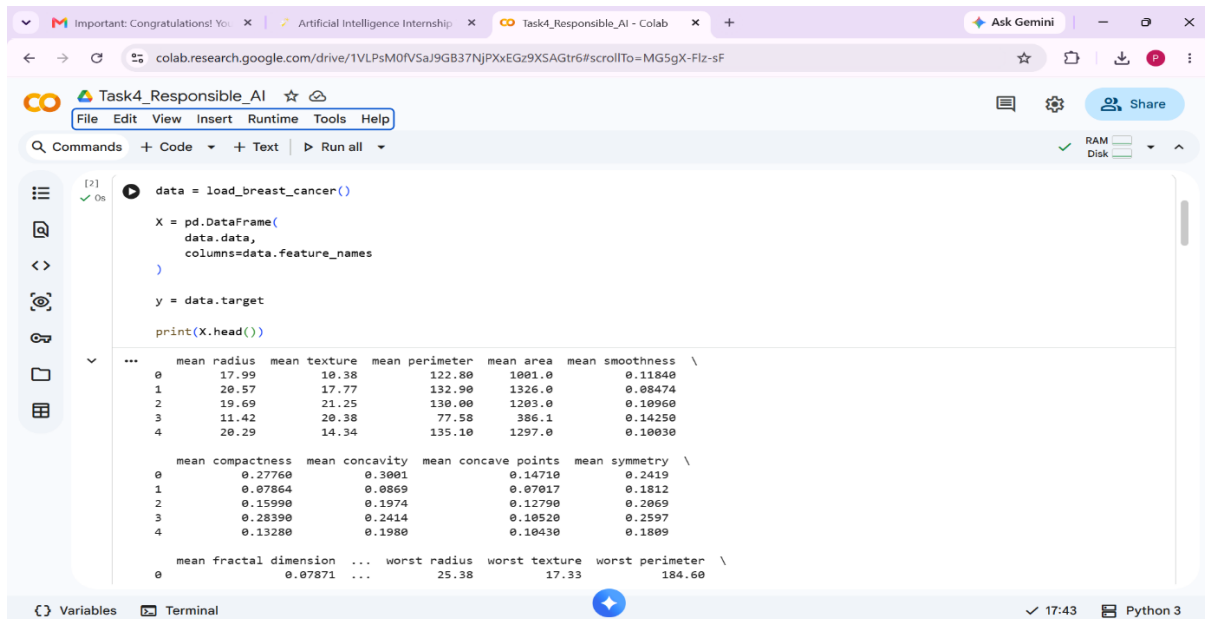
import shap
```

The output of the code cell shows the first five rows of the breast cancer dataset:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1283.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

2. Loading the Dataset

The Breast Cancer dataset was loaded into the notebook. The dataset contains several medical measurements that are used to classify whether a tumor is benign or malignant.



The screenshot shows a Google Colab notebook titled "Task4_Responsible_AI". The code cell [2] contains the following code:

```
[2] data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = data.target

print(X.head())
```

The output of the code cell shows the first five rows of the breast cancer dataset, including the target variable:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1283.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

The output also shows the first five rows of the target variable:

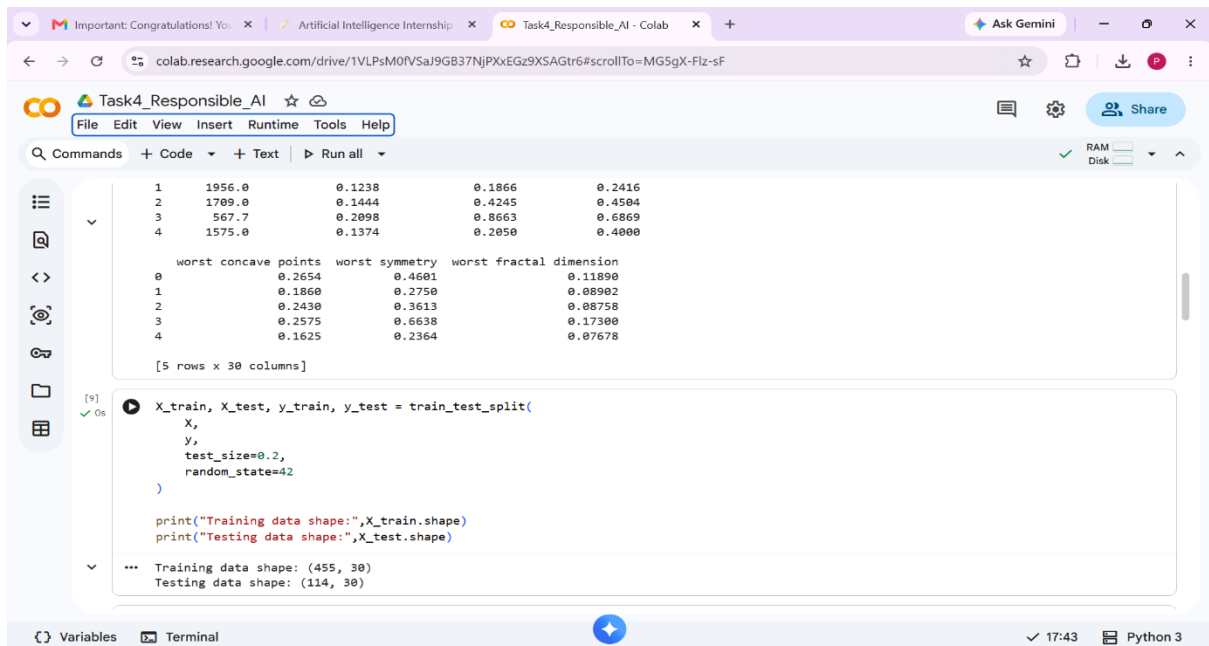
	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

The output also shows the first five rows of the dataset, including the target variable:

	mean fractal dimension	...	worst radius	worst texture	worst perimeter	\
0	0.07871	...	25.38	17.33	184.60	

3. Splitting the Dataset

The dataset was divided into training and testing data. Training data was used to train the model while testing data was used to evaluate prediction performance.



This screenshot shows a Google Colab notebook titled 'Task4_Responsible_AI'. The first code cell displays a dataset with 5 rows and 30 columns. The columns are grouped into three sets of 10: 'worst concave points', 'worst symmetry', and 'worst fractal dimension'. The second code cell uses `train_test_split` to divide the data into training and testing sets with a 0.2 test size and a random state of 42. The output shows the training data shape as (455, 30) and the testing data shape as (114, 30).

```
1 1956.0      0.1238      0.1866      0.2416
2 1709.0      0.1444      0.4245      0.4504
3 567.7       0.2098      0.8663      0.6869
4 1575.0      0.1374      0.2050      0.4000

worst concave points worst symmetry worst fractal dimension
0      0.2654      0.4601      0.11890
1      0.1860      0.2750      0.08902
2      0.2430      0.3613      0.08758
3      0.2575      0.6638      0.17300
4      0.1625      0.2364      0.07678

[5 rows x 30 columns]
```

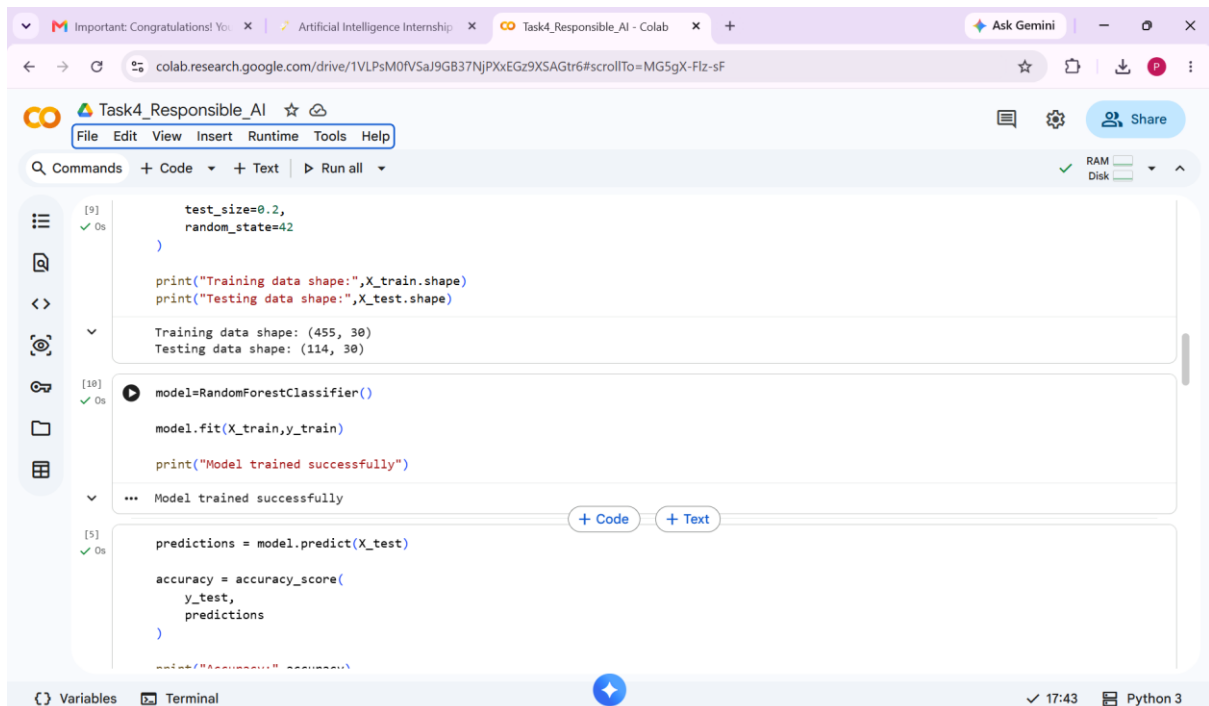
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
... Training data shape: (455, 30)
Testing data shape: (114, 30)
```

4. Training the Random Forest Model

The Random Forest algorithm was used to train the model. This algorithm builds multiple decision trees and combines their outputs to improve prediction accuracy.



This screenshot continues the Colab notebook from the previous step. The third code cell prints the training and testing data shapes. The fourth code cell creates a `RandomForestClassifier` model, fits it to the training data, and prints a success message. The fifth code cell uses the trained model to make predictions on the testing data and calculates the accuracy score.

```
test_size=0.2,
random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (455, 30)
Testing data shape: (114, 30)
```

```
model=RandomForestClassifier()

model.fit(X_train, y_train)

print("Model trained successfully")
```

```
... Model trained successfully
```

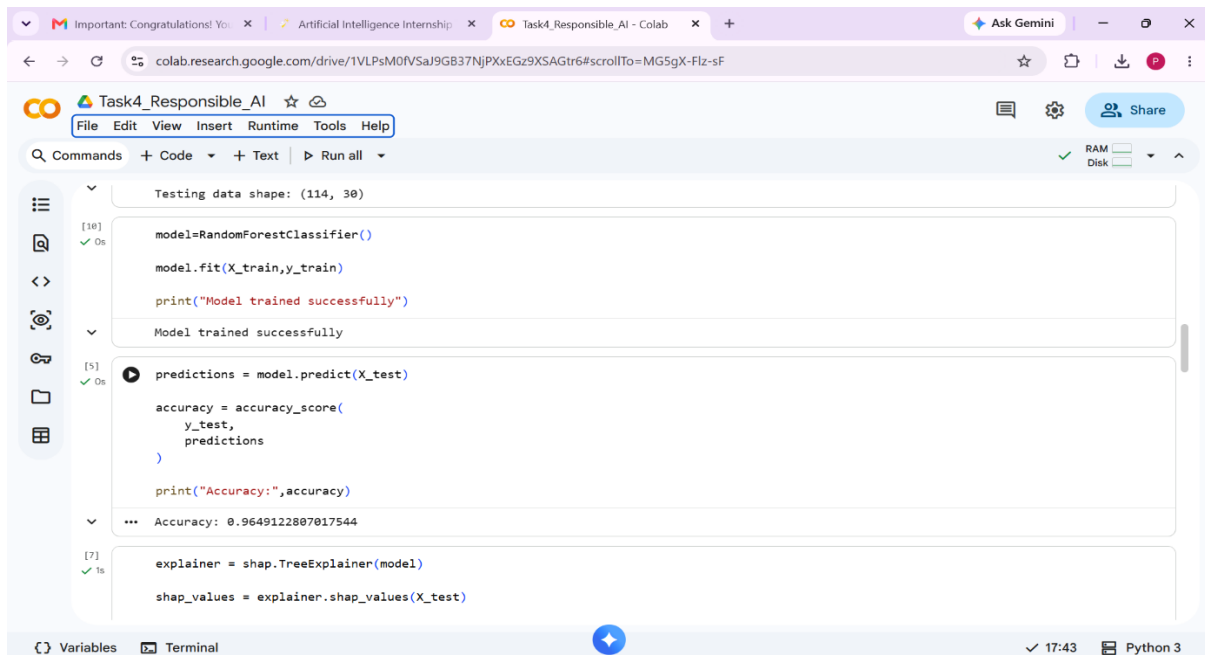
```
predictions = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    predictions
)
```

```
print("Accuracy: ", accuracy)
```

5. Evaluating Model Accuracy

Model performance was evaluated using accuracy score. The obtained result showed that the model was able to classify the dataset with high accuracy.



```
Testing data shape: (114, 30)

[10] ✓ 0s
model=RandomForestClassifier()
model.fit(X_train,y_train)
print("Model trained successfully")

Model trained successfully

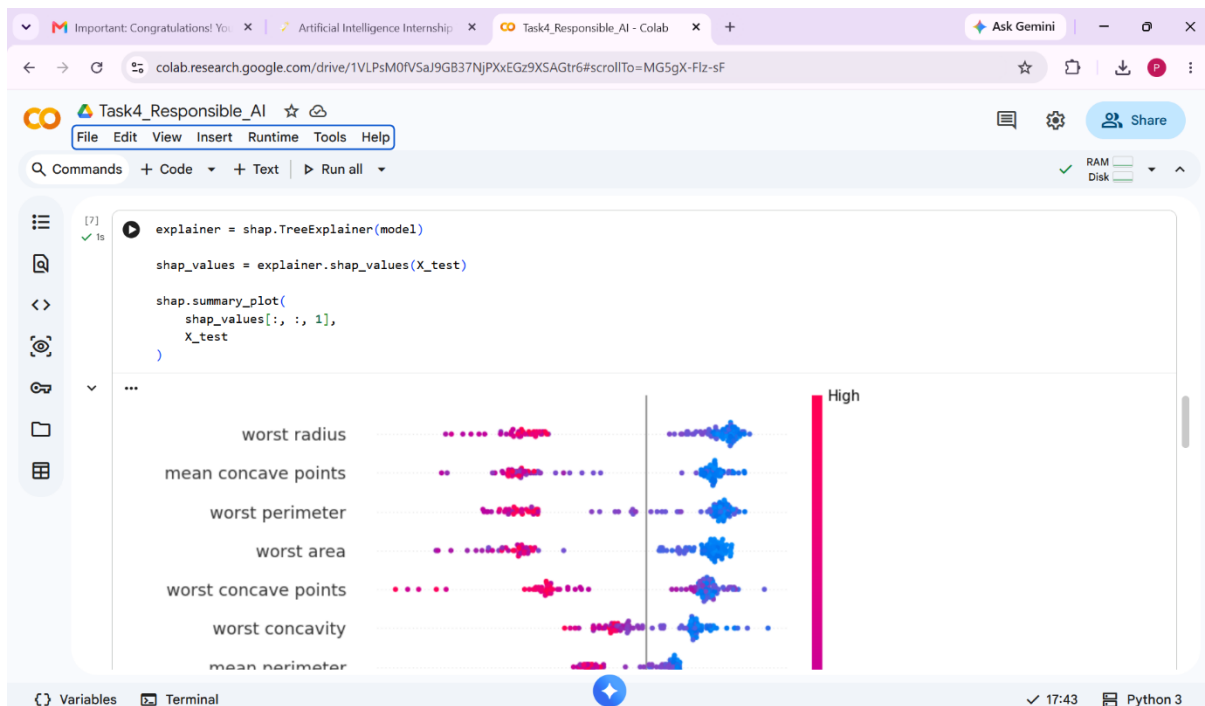
[5] ✓ 0s
predictions = model.predict(X_test)
accuracy = accuracy_score(
    y_test,
    predictions
)
print("Accuracy:",accuracy)

... Accuracy: 0.9649122807017544

[7] ✓ 1s
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```

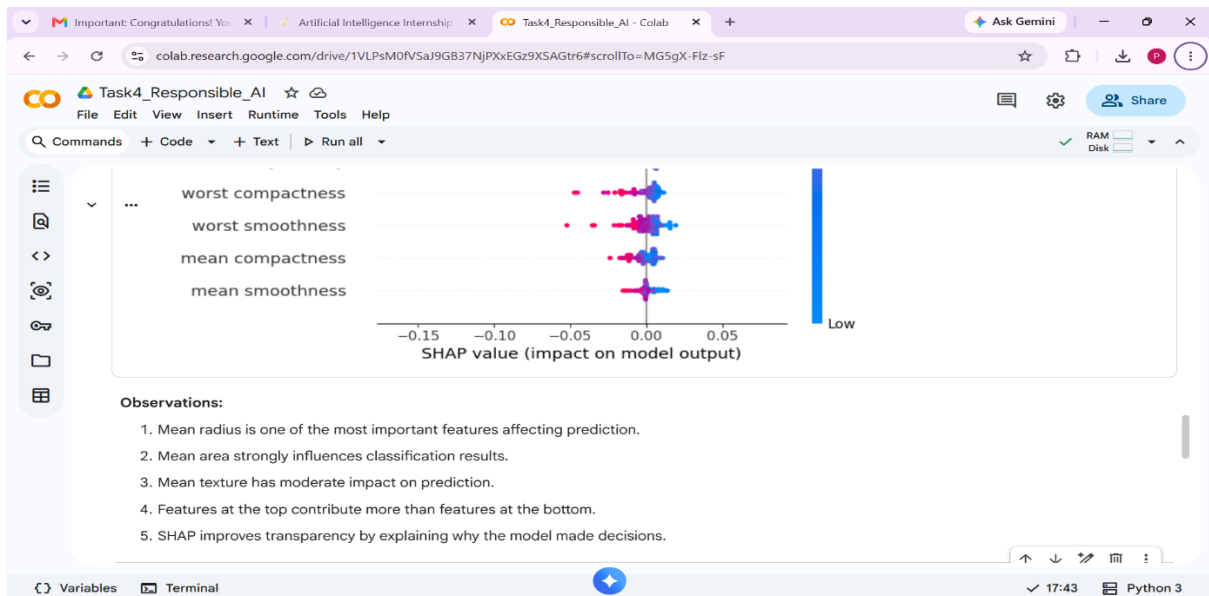
6. Feature Importance Analysis Using SHAP

SHAP analysis was applied to identify the importance of different features affecting predictions. The SHAP summary plot helps visualize the influence of each feature.



7. Observation Analysis

The generated SHAP graph was analyzed to understand how features contribute to model predictions. Features with larger effects were considered more influential.



8. Bias Analysis

Bias checking was performed by creating sample groups and comparing prediction outputs. This analysis helps determine whether predictions vary significantly across groups.

The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
[8] gender=np.random.choice(['Male','Female'], len(X_test))

results=pd.DataFrame({'Gender':gender, 'Prediction':predictions})

print(results.groupby('Gender').Prediction.mean())
```

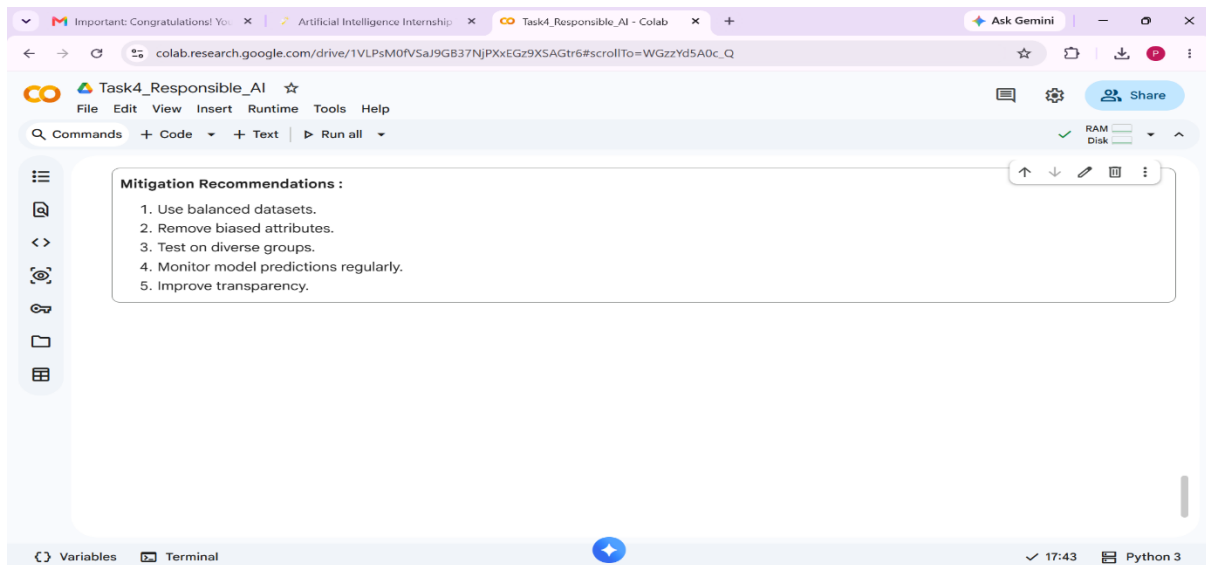
The output of the code is displayed below the code cell:

```
Gender
Female    0.612245
Male      0.661538
Name: Prediction, dtype: float64
```

The bottom of the notebook shows the 'Variables' and 'Terminal' tabs, with a status bar indicating '17:43' and 'Python 3'.

9. Mitigation Recommendations

Several mitigation methods were suggested to improve fairness and transparency in the machine learning model.



RESULTS

The Random Forest model achieved approximately 96.49% accuracy. SHAP analysis successfully identified important features influencing predictions. Bias analysis indicated only small differences between groups, showing reasonable fairness.

CONCLUSION

This project successfully demonstrated responsible AI concepts using machine learning and SHAP analysis. The model achieved high prediction accuracy while maintaining transparency through explainable AI techniques. Bias analysis and mitigation recommendations further supported the development of fair and responsible AI systems.

REFERENCES

1. Python Documentation
2. Scikit-learn Documentation
3. SHAP Documentation
4. Google Colab Documentation